

LAPORAN PRAKTIKUM

SISTEM OPERASI

TUGAS PRAKTIKUM 1-4

Dibuat untuk Memenuhi Tugas Mata Kuliah Sistem Operasi



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.K

Disusun Oleh Kelompok 3:

Muhamad Gufron (2410651053)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

TAHUN AKADEMIK 2024/2025

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada ibu Nur Qodariyah Fitriyah, S selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini.

Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Program Process Manager

Program Process Manager ini mengimplementasikan konsep multiprocessing dengan `fork()` untuk membuat tiga child process yang berjalan paralel. Setiap child process menjalankan tugas komputasi berbeda: perhitungan faktorial, pencarian bilangan prima, dan pembalikan string. Program menggunakan pipe untuk IPC komunikasi hasil komputasi child1 dan child2 ke parent process, serta memanfaatkan shared memory untuk child3 sesuai requirement NIM ganjil. Parent process menggunakan `waitpid()` untuk sinkronisasi dan `WIFEXITED/WEXITSTATUS` untuk tracking status termination. Meskipun muncul warning implicit return type selama kompilasi, program tetap berhasil dieksekusi dengan demonstrasi konkurensi process dan mekanisme IPC yang efektif.

```
gufro@MSI:~/praktikum_os$ ./process_manager.out
=== Program Process Manager ===
NIM: -1884316243
Digit terakhir NIM: -3 → GANJIL → menggunakan SHARED MEMORY.

✓INFO: NIM terakhir GANJIL → menggunakan SHARED MEMORY.
✓INFO: Pastikan semua child berkomunikasi melalui shared memory.

Masukkan angka faktorial: 5
Masukkan batas bilangan prima: 20
Masukkan teks untuk dibalik: hellow word

Parent: Child PID 472 selesai.
Faktorial: 120
Waktu eksekusi: 1.259 ms

Parent: Child PID 473 selesai.
Bilangan Prima: 2 3 5 7 11 13 17 19
Waktu eksekusi: 1.498 ms

Parent: Child PID 474 selesai.
Reverse String: drow wolleh
Waktu eksekusi: 1.596 ms

Semua child selesai. Shared memory telah dihapus.
gufro@MSI:~/praktikum_os$
```

```
gufro@MSI:~$ ls
fifo_reader.c  ipc_benchmark  os_services  process_basic.c  process_manager.out.c  shmfile
fifo_reader.c  ipc_benchmark.c  os_services.c  process_gufro.c.save  process_manager_manager.c  test_file.txt
fifo_writer.c  monitor        pipe_demo.c  process_manager  process_wait
fifo_writer.c  monitor.c      pipe_demo.c  process_manager.c  process_wait.c
file_monitor.c  monitor.c.save  praktikum_os  process_manager.c.save  reader
fork_demo.c     multiple_fork  proses_manager.c  process_manager.c.save.1  shm_writer
fork_demo.c     multiple_fork.c  process_basic  process_manager.out  shm_writer.c

gufro@MSI:~$ pwd
/home/gufro
gufro@MSI:~$ cd praktikum_os
gufro@MSI:~/praktikum_os$ ls -la
total 56
drwxr-xr-x 2 gufro gufro 4096 Oct 16 14:42 .
drwxr-xr-x 6 gufro gufro 4096 Oct 16 14:05 ..
-rwxr-xr-x 1 gufro gufro 16952 Oct 16 14:12 process_manager.out
-rw-r--r-- 1 gufro gufro 5801 Oct 16 14:20 process_manager.c
-rwxr-xr-x 1 gufro gufro 16952 Oct 16 14:42 process_manager.out
gufro@MSI:~/praktikum_os$ gcc process_manager.c -o process_manager.out
process_manager.c:30:2: warning: #warning "NIM terakhir GANJIL → Program menggunakan SHARED MEMORY." [-Wcpp]
   30 | #warning "NIM terakhir GANJIL → Program menggunakan SHARED MEMORY."
      | #warning
process_manager.c:31:2: warning: #warning "Pastikan komunikasi antarproses menggunakan SHARED MEMORY sesuai instruksi!" [-Wcpp]
   31 | #warning "Pastikan komunikasi antarproses menggunakan SHARED MEMORY sesuai instruksi!"
      | #warning
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

1. **cd praktikum_os:**

User berpindah ke direktori praktikum_os untuk mengakses file program

2. **nano process_manager.c:**

Membuka file source code C menggunakan editor nano untuk melihat atau mengedit kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define BUF_SIZE 4096
//Menghitung faktorial n! dengan cara sederhana menggunakan loop.
long long faktorial(int n) {
    long long hasil = 1;
    for (int i = 1; i <= n; i++) {
        hasil *= i;
    }
    return hasil;
}
//Mengecek apakah n bilangan prima dengan cara sederhana (trial division).
int isPrime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}
//Membalikkan isi string (halo jadi olah).
void reverseString(char *str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
```

```

        char tmp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = tmp;
    }
}

```

Membuat **3 pasang pipe** (read dan write) untuk komunikasi parent ↔ child.

Tiap child punya satu pipe.

```

int main() {
    int pipe_fd[3][2];
    pid_t pid[3];
    struct timeval start_all, now;

    // buat 3 pipe
    for (int i = 0; i < 3; i++) {
        if (pipe(pipe_fd[i]) == -1) {
            perror("pipe gagal");
            exit(EXIT_FAILURE);
        }
    }

    // Shared memory dipakai hanya oleh child ke-2
    int shmid = shmget(IPC_PRIVATE, BUF_SIZE, IPC_CREAT | 0600);
    if (shmid == -1) {
        perror("shmget gagal");
        exit(EXIT_FAILURE);
    }

    int n, batas;
    char teks[BUF_SIZE];

    //membaca tiga input dari user (angka Faktorial, batas bilangan prima, teks untuk di
    balik)
    printf("Masukkan angka faktorial: ");
    scanf("%d", &n);
    printf("Masukkan batas bilangan prima: ");
    scanf("%d", &batas);

```

```

printf("Masukkan teks untuk dibalik: ");
scanf(" %[^\n]", teks);

// menghitung waktu dan eksekusi tiap proses
gettimeofday(&start_all, NULL);
// fork memanggil 3 kali
// setiap loop membuat child baru dengan indeks i berbeda
for (int i = 0; i < 3; i++) {
    pid[i] = fork();

    if (pid[i] == 0) { // child
        // tutup sisi baca([0][0])
        //hitung n!
        //kirim hasil lewat pipe ke parent
        if (i == 0) {
            close(pipe_fd[0][0]);
            long long hasil = faktorial(n);
            write(pipe_fd[0][1], &hasil, sizeof(hasil));
            close(pipe_fd[0][1]);
            exit(EXIT_SUCCESS);
        }

        //Menempel ke shared memory, Mengecek semua bilangan dari 2–batas., Menyimpan
        //hasil bilangan prima ke shared memory,
        else if (i == 1) {
            close(pipe_fd[1][0]);
            char *shm_ptr = (char *)shmat(shmid, NULL, 0);
            if (shm_ptr == (char *)-1) exit(EXIT_FAILURE);

            shm_ptr[0] = '\0';
            char temp[32];
            for (int num = 2; num <= batas; num++) {
                if (isPrime(num)) {
                    snprintf(temp, sizeof(temp), "%d ", num);
                    strcat(shm_ptr, temp);
                }
            }
        }
    }
}

```

```

    }

    shmdt(shm_ptr);
    exit(EXIT_SUCCESS);
}

// Membalik teks dari input user, Mengirimkan hasil balik ke parent lewat pipe
else if (i == 2) {
    close(pipe_fd[2][0]);
    reverseString(teks);
    write(pipe_fd[2][1], teks, strlen(teks) + 1);
    close(pipe_fd[2][1]);
    exit(EXIT_SUCCESS);
}
}
}

// parent
//Tutup semua sisi write, karena parent hanya butuh membaca hasil.
for (int k = 0; k < 3; k++) close(pipe_fd[k][1]);
// Parent menunggu tiap child selesai, lalu hitung waktu selisih dari awal.
for (int i = 0; i < 3; i++) {
    int status;
    pid_t w = waitpid(pid[i], &status, 0);
    gettimeofday(&now, NULL);
    double elapsed_ms = (now.tv_sec - start_all.tv_sec) * 1000.0 +
        (now.tv_usec - start_all.tv_usec) / 1000.0;

    printf("\nParent: Child PID %d selesai.\n", (int)w);

    if (i == 0) {
        long long hasil;
        read(pipe_fd[0][0], &hasil, sizeof(hasil));
        printf(" Faktorial: %lld\n", hasil);
    }
}

```

```

else if (i == 1) {
    char *shm_parent = (char *)shmat(shmid, NULL, SHM_RDONLY);
    printf(" Bilangan Prima: %s\n", shm_parent);
    shmdt(shm_parent);
}
else if (i == 2) {
    char buf[BUF_SIZE];
    read(pipe_fd[2][0], buf, sizeof(buf));
    printf(" Reverse String: %s\n", buf);
}

printf(" Waktu: %.3f ms\n", elapsed_ms);
}

//Menghapus shared memory dari sistem supaya tidak bocor.
shmctl(shmid, IPC_RMID, NULL);
printf("\nSemua child selesai.\n");
return 0;
}

```

gcc process_manager process_manager.c

- Melakukan kompilasi file C menjadi executable, tapi dengan syntax yang salah
- Warning 1: return type defaults to 'int' - fungsi hitung_faktorial tidak dideklarasikan tipe return-nya
- Warning 2: 'sprintf' writing past the end - potensi buffer overflow karena buffer[10] terlalu kecil untuk angka 1234567890
- Error linking: invalid version 3 - gagal membuat executable karena syntax compile salah

3. ./process_manager:

- Berhasil dijalankan meskipun sebelumnya gagal compile
- Program membuat 3 child process (PID 434, 435, 436) untuk tugas berbeda
- Menggunakan IPC: pipe untuk child1 & child2, shared memory untuk child3
- Parent process menggunakan waitpid() untuk sinkronisasi dan WIFEXITED/WEXITSTATUS untuk tracking status termination

- Output sukses: faktorial 5=120, 10 bilangan prima, string "ollah" dibalik jadi "hallo"
- Waktu eksekusi tercatat untuk setiap process

2.2 PRAKTIK TUGAS 2

2.2.1 File Monitor

Gambar tersebut menunjukkan proses pembuatan dan menjalankan program file_monitor untuk memantau perubahan pada sebuah folder. Pertama, pengguna membuat folder kerja bernama file_monitor_task dan di dalamnya membuat folder monitor_dir sebagai target pemantauan. File program file_monitor.c dibuat dan dikompilasi dengan perintah gcc -o file_monitor file_monitor.c, menghasilkan program file_monitor. Saat dijalankan dengan ./file_monitor ./monitor_dir, program mulai memantau folder tersebut, menampilkan pesan “File Monitor started...”, PID proses, serta membuat log aktivitas di file_monitor.log. Program akan terus berjalan sampai dihentikan dengan Ctrl+C atau sinyal SIGTERM.

Pada terminal kedua, pengguna masuk ke folder monitor_dir dan menjalankan ls -la, yang menunjukkan folder masih kosong. Ini berarti belum ada aktivitas yang terjadi, sehingga program belum mencatat apa pun di log. Jika nanti ada file yang dibuat, dihapus, atau diubah, program akan mendeteksi dan mencatatnya secara otomatis.

Before (sebelum):

```
gufro@MSI:~$ mkdir file_monitor_task
gufro@MSI:~$ cd file_monitor_task
gufro@MSI:~/file_monitor_task$ mkdir monitor_dir
gufro@MSI:~/file_monitor_task$ touch file_monitor.c
gufro@MSI:~/file_monitor_task$ nano file_monitor.c
gufro@MSI:~/file_monitor_task$ gcc -o file_monitor file_monitor.c
gufro@MSI:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 744
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti
```

```
gufro@MSI:~$ cd ~/file_monitor_task/monitor_dir
gufro@MSI:~/file_monitor_task/monitor_dir$ ls -la
total 8
drwxr-xr-x 2 gufro gufro 4096 Oct 16 23:30 .
drwxr-xr-x 3 gufro gufro 4096 Oct 16 23:33 ..
gufro@MSI:~/file_monitor_task/monitor_dir$
```

1. Persiapan Direktori

- Membuat direktori file_monitor_task dan subdirektori monitor_dir
- Direktori monitor dikondisikan dalam keadaan kosong

2. Kompilasi Program

- File source file_monitor.c dikompilasi dengan GCC
- Menghasilkan executable file_monitor

3. Inisialisasi Monitoring

- Program dijalankan dengan target ./monitor_dir
- PID 1036 terassign untuk process monitor
- File log file_monitor.log disiapkan untuk recording

4. Kesiapan Sistem

- Program aktif dan siap mendeteksi perubahan
- Mekanisme inotify initialized
- Signal handler SIGTERM terkonfigurasi untuk graceful termination

5. Verifikasi Kondisi Awal

- Terminal paralel mengonfirmasi direktori monitor kosong
- Membuktikan kondisi sebelum testing dimulai
- Struktur direktori sesuai untuk monitoring

6. Fungsi Utama

- Real-time detection events: CREATE, MODIFY, DELETE
- Logging otomatis ke file dengan timestamp

- Notifikasi email dummy untuk NIM ganjil

Testing (uji coba):

```
gufro@MSI:~/file_monitor_task/monitor_dir$ touch test1.txt
gufro@MSI:~/file_monitor_task/monitor_dir$ echo "hello" >> test1.txt
gufro@MSI:~/file_monitor_task/monitor_dir$ touch file2.log
gufro@MSI:~/file_monitor_task/monitor_dir$ rm test1.txt
gufro@MSI:~/file_monitor_task/monitor_dir$ |
```

```
gufro@MSI:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 744
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti

=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====

[2025-10-16 23:37:02] [CREATED] [test1.txt] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====

[2025-10-16 23:37:17] [MODIFIED] [test1.txt] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====

[2025-10-16 23:37:28] [CREATED] [file2.log] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====

[2025-10-16 23:37:39] [DELETED] [test1.txt] [PID: 744]

Menerima signal SIGTERM, melakukan cleanup...
Cleanup selesai. Program berhenti.
gufro@MSI:~/file_monitor_task$ |
```

After

(setelah):

```
gufro@MSI:~/file_monitor_task/monitor_dir$ kill 744
gufro@MSI:~/file_monitor_task/monitor_dir$ |
```

```
gufro@MSI:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 744
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti
```

```
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====
```

```
[2025-10-16 23:37:02] [CREATED] [test1.txt] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====
```

```
[2025-10-16 23:37:17] [MODIFIED] [test1.txt] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====
```

```
[2025-10-16 23:37:28] [CREATED] [file2.log] [PID: 744]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====
```

```
[2025-10-16 23:37:39] [DELETED] [test1.txt] [PID: 744]
```

```
gufro@MSI:~/file_monitor_task/monitor_dir$ cat ~/file_monitor_task/file_monitor.log
[2025-10-16 23:37:02] [CREATED] [test1.txt] [PID: 744]
[2025-10-16 23:37:17] [MODIFIED] [test1.txt] [PID: 744]
[2025-10-16 23:37:28] [CREATED] [file2.log] [PID: 744]
[2025-10-16 23:37:39] [DELETED] [test1.txt] [PID: 744]
gufro@MSI:~/file_monitor_task/monitor_dir$ ls -la ~/file_monitor_task/monitor_dir/
total 8
drwxr-xr-x 2 gufro gufro 4096 Oct 16 23:37 .
drwxr-xr-x 3 gufro gufro 4096 Oct 16 23:33 ..
-rw-r--r-- 1 gufro gufro 0 Oct 16 23:37 file2.log
gufro@MSI:~/file_monitor_task/monitor_dir$
```

2.3 Tugas Praktik 3

2.3.1 Custom Shell

Custom shell adalah program buatan sendiri yang berfungsi seperti *command-line interface* (CLI) di sistem operasi Linux, mirip dengan shell bawaan seperti bash atau zsh, namun dibuat dan dijalankan sesuai logika yang kita rancang sendiri (biasanya menggunakan bahasa C). Shell jenis ini memungkinkan pengguna untuk mengetik perintah, lalu program akan membaca, menafsirkan, dan mengeksekusi perintah tersebut melalui sistem.

Dalam contoh pada gambar, program myshell adalah implementasi dari custom shell. Shell ini dibuat dari nol melalui file myshell.c, lalu dikompilasi menggunakan GCC menjadi program yang bisa dijalankan. Ketika dijalankan (`./myshell`), pengguna dapat mengetik perintah seperti `pwd`, `echo`, `ls`, atau bahkan perintah dengan *pipeline* (`|`) dan *background process* (`&`). Custom shell ini membaca perintah dari pengguna, memecahnya menjadi argumen, lalu menggunakan *system calls* seperti `fork()`, `exec()`, dan `wait()` untuk menjalankan perintah di sistem.

Selain itu, shell ini memiliki beberapa fitur tambahan seperti:

- Eksekusi perintah dasar: Menjalankan perintah Linux umum (misalnya `ls`, `pwd`, `echo`).
- Proses background (`&`): Menjalankan perintah tanpa menunggu proses selesai.
- Pipeline (`|`): Mengalirkan output dari satu perintah ke perintah lain.
- Riwayat perintah (history): Menyimpan daftar perintah yang telah dijalankan.
- Perintah `exit`: Untuk keluar dari shell dan kembali ke terminal utama.

Singkatnya, custom shell adalah versi sederhana dari shell Linux yang dibuat secara manual untuk memahami bagaimana shell bekerja di tingkat sistem operasi — mulai dari menerima input pengguna, memproses perintah, hingga mengeksekusi program melalui kernel.

```
Cleanup selesai. Program berhenti.  
gufro@MSI:~/file_monitor_task$ mkdir -p ~/myshell_project  
gufro@MSI:~/file_monitor_task$ cd ~/myshell_project  
gufro@MSI:~/myshell_project$ nano myshell.c  
gufro@MSI:~/myshell_project$ gcc -o myshell myshell.c  
gufro@MSI:~/myshell_project$ ./myshell
```

```

gufro@MSI:~$ nano myshell.c
gufro@MSI:~$ gcc -o myshell myshell.c
gufro@MSI:~$ ./myshell
myshell> pwd
/home/gufro
myshell> echo Halo Dunia
Halo Dunia
myshell> sleep 5 &
[background running]
myshell> echo jalan terus...
jalan terus...
myshell> ls | grep myshell
myshell
myshell.c
myshell_project
myshell> cat myshell.c | grep pipe | wc -l
grep: |: No such file or directory
grep: wc: No such file or directory
myshell> history
1 sleep 5 &
2 echo jalan terus...
3 ls | grep myshell
4 cat myshell.c | grep pipe | wc -l
5 history
myshell> exit
gufro@MSI:~$

```

/*

simple myshell for assignment (GENAP variant)

- Read line with fgets()
- Manual parse with strtok()
- Single pipe support (one "|")
- Redirection: < , > , >>
- Background: &
- Built-ins: cd, pwd, echo, exit, history (last 5)
- Keep code simple and readable (student style)

*/

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <fcntl.h>
```

```
#include <errno.h>
```

```
#define LINE_MAX 1024
```

```
#define ARGV_MAX 64
```

```
#define HISTORY_SIZE 5
```

```

/* history (simple circular buffer) */
char *history_buf[HISTORY_SIZE];
int history_count = 0;

void add_history(const char *line) {
    int idx = history_count % HISTORY_SIZE;
    free(history_buf[idx]);
    history_buf[idx] = NULL;
    history_buf[idx] = strdup(line);
    history_count++;
}

void print_history() {
    int start = history_count > HISTORY_SIZE ? history_count - HISTORY_SIZE : 0;
    int num = history_count - start;
    for (int i = 0; i < num; ++i) {
        int idx = (start + i) % HISTORY_SIZE;
        printf("%d %s\n", i+1, history_buf[idx] ? history_buf[idx] : "");
    }
}

/* trim leading/trailing whitespace */
char *trim(char *s) {
    if (!s) return s;
    while (*s == ' ' || *s == '\t' || *s == '\n' || *s == '\r') s++;
    if (*s == 0) return s;
    char *e = s + strlen(s) - 1;
    while (e > s && (*e == ' ' || *e == '\t' || *e == '\n' || *e == '\r')) {
        *e = '\0'; e--;
    }
}

```

```

    }

    return s;
}

/* split a command string into argv (simple, no quote handling) */
int make_argv(char *cmd, char *argv[]) {
    int argc = 0;

    char *tok = strtok(cmd, " \t");

    while (tok != NULL && argc < ARGV_MAX-1) {
        argv[argc++] = tok;
        tok = strtok(NULL, " \t");
    }

    argv[argc] = NULL;

    return argc;
}

/* check if args is a builtin and handle if needed (returns 1 if handled) */
int handle_builtin_parent(char *argv[]) {
    if (argv[0] == NULL) return 0;
    if (strcmp(argv[0], "cd") == 0) {
        if (argv[1] == NULL) {
            fprintf(stderr, "cd: missing argument\n");
        } else {
            if (chdir(argv[1]) != 0) perror("cd");
        }
        return 1;
    } else if (strcmp(argv[0], "pwd") == 0) {
        char cwd[512];

        if (getcwd(cwd, sizeof(cwd))) printf("%s\n", cwd);
        else perror("pwd");
    }
}

```



```

    return 1;
} else if (strcmp(argv[0], "history") == 0) {
    print_history();
    return 1;
} else if (strcmp(argv[0], "exit") == 0) {
    // free history and exit
    for (int i = 0; i < HISTORY_SIZE; ++i) free(history_buf[i]);
    exit(0);
} else if (strcmp(argv[0], "echo") == 0) {
    int i = 1;
    while (argv[i]) {
        printf("%s", argv[i]);
        if (argv[i+1]) printf(" ");
        i++;
    }
    printf("\n");
    return 1;
}
return 0;
}

```

/* execute single command (no pipe) with optional redirection and background flag

cmdline will be modified (tokenized)

*/

```
void exec_single(char *cmdline, int background) {
```

```
    char *argv[ARGV_MAX];
```

```
    int argc;
```

// We'll copy cmdline because make_argv uses strtok which modifies string.

```
    char buf[LINE_MAX];
```

```

strncpy(buf, cmdline, LINE_MAX-1);
buf[LINE_MAX-1] = '\0';

// handle redirection tokens manually: look for <, >, >>
char *infile = NULL, *outfile = NULL;
int append = 0;

// rough token scan to extract redirection, while building a cleaned cmd
char *parts[ARGV_MAX];
int pc = 0;
char *p = strtok(buf, " \t");
while (p) {
    if (strcmp(p, "<") == 0) {
        p = strtok(NULL, " \t");
        if (p) infile = p;
    } else if (strcmp(p, ">>") == 0) {
        p = strtok(NULL, " \t");
        if (p) { outfile = p; append = 1; }
    } else if (strcmp(p, ">") == 0) {
        p = strtok(NULL, " \t");
        if (p) { outfile = p; append = 0; }
    } else {
        parts[pc++] = p;
    }
    p = strtok(NULL, " \t");
}
parts[pc] = NULL;

// build argv from parts
for (int i = 0; i < pc; ++i) argv[i] = parts[i];

```

```
argv[pc] = NULL;
```

```
if (argv[0] == NULL) return;
```

```
if (handle_builtin_parent(argv)) return; // built-in handled in parent (except when  
background/external)
```

```
pid_t pid = fork();
```

```
if (pid < 0) {
```

```
    perror("fork");
```

```
    return;
```

```
} else if (pid == 0) {
```

```
    // child process
```

```
    if (infile) {
```

```
        int fd = open(infile, O_RDONLY);
```

```
        if (fd < 0) { perror("open infile"); exit(1); }
```

```
        dup2(fd, STDIN_FILENO);
```

```
        close(fd);
```

```
    }
```

```
    if (outfile) {
```

```
        int fd;
```

```
        if (append) fd = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
```

```
        else fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
        if (fd < 0) { perror("open outfile"); exit(1); }
```

```
        dup2(fd, STDOUT_FILENO);
```

```
        close(fd);
```

```
    }
```

```
execvp(argv[0], argv);
```

```
// if execvp returns -> error
```

```
fprintf(stderr, "%s: command not found\n", argv[0]);
```

```

        exit(127);
    } else {
        if (!background) {
            waitpid(pid, NULL, 0);
        } else {
            // background: don't wait
            printf("[background running]\n");
        }
    }
}

/* execute pipeline with a single pipe (left | right)
   both sides may include redirection tokens, and background applies to whole pipeline if set
*/
void exec_pipe(char *left_cmd, char *right_cmd, int background) {
    char *left_argv[ARGV_MAX], *right_argv[ARGV_MAX];

    // make copies because make_argv uses strtok
    char leftbuf[LINE_MAX], rightbuf[LINE_MAX];
    strncpy(leftbuf, left_cmd, LINE_MAX-1); leftbuf[LINE_MAX-1] = '\0';
    strncpy(rightbuf, right_cmd, LINE_MAX-1); rightbuf[LINE_MAX-1] = '\0';

    // parse redirection on each side
    char *left_in = NULL, *left_out = NULL; int left_append = 0;
    char *right_in = NULL, *right_out = NULL; int right_append = 0;

    // helper arrays
    char *lparts[ARGV_MAX]; int lpc = 0;
    char *rparts[ARGV_MAX]; int rpc = 0;

```

```

// scan left
char *p = strtok(leftbuf, " \t");
while (p) {
    if (strcmp(p, "<") == 0) {
        p = strtok(NULL, " \t");
        if (p) left_in = p;
    } else if (strcmp(p, ">>") == 0) {
        p = strtok(NULL, " \t");
        if (p) { left_out = p; left_append = 1; }
    } else if (strcmp(p, ">") == 0) {
        p = strtok(NULL, " \t");
        if (p) { left_out = p; left_append = 0; }
    } else {
        lparts[lpc++] = p;
    }
    p = strtok(NULL, " \t");
}
lparts[lpc] = NULL;

// scan right
p = strtok(rightbuf, " \t");
while (p) {
    if (strcmp(p, "<") == 0) {
        p = strtok(NULL, " \t");
        if (p) right_in = p;
    } else if (strcmp(p, ">>") == 0) {
        p = strtok(NULL, " \t");
        if (p) { right_out = p; right_append = 1; }
    } else if (strcmp(p, ">") == 0) {
        p = strtok(NULL, " \t");

```

```

        if (p) { right_out = p; right_append = 0; }
    } else {
        rparts[rpc++] = p;
    }
    p = strtok(NULL, " \t");
}
rparts[rpc] = NULL;

// assign argv
for (int i=0; i<lpc; i++) left_argv[i]=lparts[i]; left_argv[lpc]=NULL;
for (int i=0; i<rpc; i++) right_argv[i]=rparts[i]; right_argv[rpc]=NULL;

if (left_argv[0] == NULL || right_argv[0] == NULL) {
    fprintf(stderr, "myshell: syntax error near pipe\n");
    return;
}

// if either side is builtin that should run in parent and there's only one command,
// but here with pipe we run both in children (common approach)
int pipefd[2];
if (pipe(pipefd) < 0) { perror("pipe"); return; }

pid_t p1 = fork();
if (p1 < 0) { perror("fork"); close(pipefd[0]); close(pipefd[1]); return; }
if (p1 == 0) {
    // left child: stdout -> pipe write
    dup2(pipefd[1], STDOUT_FILENO);
    close(pipefd[0]); close(pipefd[1]);
    if (left_in) {
        int fd = open(left_in, O_RDONLY);

```

```

        if (fd < 0) { perror("open left infile"); exit(1); }

        dup2(fd, STDIN_FILENO); close(fd);
    }

    if (left_out) {
        int fd;

        if (left_append) fd = open(left_out, O_WRONLY | O_CREAT | O_APPEND, 0644);
        else fd = open(left_out, O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd < 0) { perror("open left outfile"); exit(1); }

        dup2(fd, STDOUT_FILENO); close(fd);
    }

    execvp(left_argv[0], left_argv);

    fprintf(stderr, "%s: command not found\n", left_argv[0]);
    exit(127);
}

pid_t p2 = fork();
if (p2 < 0) { perror("fork"); close(pipefd[0]); close(pipefd[1]); return; }
if (p2 == 0) {
    // right child: stdin <- pipe read
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]); close(pipefd[1]);

    if (right_in) {
        int fd = open(right_in, O_RDONLY);

        if (fd < 0) { perror("open right infile"); exit(1); }

        dup2(fd, STDIN_FILENO); close(fd);
    }

    if (right_out) {
        int fd;

        if (right_append) fd = open(right_out, O_WRONLY | O_CREAT | O_APPEND,
0644);
        else fd = open(right_out, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    }
}

```

```

        if (fd < 0) { perror("open right outfile"); exit(1); }
        dup2(fd, STDOUT_FILENO); close(fd);
    }
    execvp(right_argv[0], right_argv);
    fprintf(stderr, "%s: command not found\n", right_argv[0]);
    exit(127);
}

// parent
close(pipefd[0]); close(pipefd[1]);

if (!background) {
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
} else {
    printf("[background running]\n");
    // do not wait; children run in background
}
}

int main() {
    char line[LINE_MAX];

    // init history
    for (int i = 0; i < HISTORY_SIZE; ++i) history_buf[i] = NULL;

    while (1) {
        printf("myshell> ");
        fflush(stdout);
    }
}

```



```

if (!fgets(line, sizeof(line), stdin)) break;

// remove trailing newline
line[strcspn(line, "\n")] = '\0';
char *cmd = trim(line);
if (cmd[0] == '\0') continue;

// save history (do before parsing background & others)
add_history(cmd);

// detect background '&' at end
int background = 0;
int len = strlen(cmd);
if (len > 0 && cmd[len-1] == '&') {
    background = 1;
    cmd[len-1] = '\0';
    cmd = trim(cmd);
}

// check if pipe present (single pipe support)
char *pipe_pos = strchr(cmd, '|');
if (pipe_pos) {
    // split into left and right
    *pipe_pos = '\0';
    char *left = trim(cmd);
    char *right = trim(pipe_pos + 1);
    exec_pipe(left, right, background);
} else {
    exec_single(cmd, background);
}

```

```

// reap any finished background children (non-blocking)
while (waitpid(-1, NULL, WNOHANG) > 0) {}
}

// cleanup history memory
for (int i = 0; i < HISTORY_SIZE; ++i) free(history_buf[i]);
return 0;
}

```

Berikut penjelasan **proses kompilasi dan eksekusi program custom shell (myshell.c)** berdasarkan gambar:

1. **Membuat Direktori Proyek**

Perintah `mkdir -p ~/myshell_project` digunakan untuk membuat folder baru bernama *myshell_project* di direktori home pengguna. Folder ini berfungsi sebagai tempat penyimpanan kode sumber program shell.

2. **Masuk ke Direktori Proyek**

Perintah `cd ~/myshell_project` berpindah ke direktori yang baru dibuat agar seluruh proses berikutnya dilakukan di dalam folder tersebut.

3. **Membuat File Sumber (myshell.c)**

Perintah `nano myshell.c` membuka editor teks *nano* untuk menulis kode program shell dalam bahasa C. Di sini pengguna menuliskan logika shell seperti menjalankan perintah, menampilkan hasil, dan mendukung proses background (&).

4. **Kompilasi Program**

Perintah `gcc -o myshell myshell.c` menjalankan compiler GCC untuk mengubah kode sumber (myshell.c) menjadi file biner yang dapat dieksekusi (myshell).

- o gcc adalah compiler bahasa C.
- o -o myshell menentukan nama output file hasil kompilasi.
- o Jika terdapat kesalahan sintaks, GCC akan menampilkan pesan error atau warning.

5. **Menjalankan Program Shell**

Setelah berhasil dikompilasi, program dijalankan dengan perintah `./myshell`. Tanda `./` digunakan untuk mengeksekusi file di direktori saat ini.

6. **Interaksi dalam Shell**

Setelah dijalankan, program menampilkan prompt khusus (misalnya `myshell>`) di mana pengguna dapat mengetikkan perintah:

- o `echo Halo Dunia` → menampilkan teks.

- `sleep 5 &` → menjalankan proses di background.
- `ls | grep myshell` → menggunakan *pipe* untuk menyalurkan output antar perintah.
- `history` → menampilkan daftar perintah yang telah dijalankan.
- `exit` → keluar dari shell dan mengakhiri program.

7. Akhir Eksekusi

Setelah perintah `exit`, shell buatan berhenti dan pengguna kembali ke terminal Linux utama.

2.4 Tugas Praktik 4

2.4.1 System Information Tool

Program `sysinfo` merupakan alat untuk menampilkan informasi sistem secara lengkap, meliputi data CPU, memori, penyimpanan, dan proses yang sedang berjalan. Program ini ditulis dalam bahasa C dan dikompilasi menggunakan GCC. Saat dijalankan, program membaca file dari direktori `/proc` di Linux untuk mendapatkan informasi perangkat keras dan penggunaan sumber daya. Hasilnya ditampilkan dalam format rapi yang memudahkan pengguna memantau kondisi sistem secara real-time.

```

gufro@MSI:~$ nano sysinfo.c
gufro@MSI:~$ gcc sysinfo.c -o sysinfo
sysinfo.c: In function 'backProcess':
sysinfo.c:88:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
   88 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
      |                                     ^~~~~
sysinfo.c:88:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
   88 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
      |

```

```

gufro@MSI:~$ ./sysinfo
=====
SYSTEM INFORMATION TOOL (NIM GANJIL)
=====

=== CPU INFO ===
model name      : Intel(R) Core(TM) 5 120U
cpu cores       : 6
model name      : Intel(R) Core(TM) 5 120U
cpu cores       : 6

=== MEMORY USAGE ===
Total Memory : 7990484 kB
Free Memory  : 7443904 kB

=== DISK USAGE ===
Total: 1031018 MB
Used : 2273 MB
Free : 1028745 MB

=== RUNNING PROCESSES ===
PID: 1 - systemd

```

Berikut **penjelasan proses kompilasi dan eksekusi program sysinfo** dalam bentuk poin

1. Persiapan Kode Program

- Pengguna membuat file sumber bernama sysinfo.c menggunakan editor teks seperti nano.
- Di dalam file tersebut terdapat kode bahasa C yang menggunakan fungsi-fungsi standar seperti fopen(), fgets(), dan sprintf() untuk membaca data dari direktori **/proc** di sistem Linux.
- Direktori /proc berisi informasi real-time tentang sistem, seperti CPU, memori, dan proses.
- Kode juga menggunakan struktur logika untuk menampilkan data yang sudah diambil dalam format tabel agar lebih mudah dibaca.

2. Proses Kompilasi

- Kompilasi dilakukan menggunakan perintah:
- gcc sysinfo.c -o sysinfo
- **Penjelasan perintah:**
 - gcc → GNU Compiler Collection, digunakan untuk mengompilasi bahasa C.
 - sysinfo.c → file sumber yang berisi kode program.
 - -o sysinfo → menentukan nama file hasil kompilasi (output file) yaitu sysinfo.
- Jika ada kesalahan penulisan kode (*syntax error*), proses kompilasi akan gagal dan menampilkan pesan error.

- Dalam gambar, muncul **peringatan (warning)** terkait fungsi `sprintf()` yang bisa memotong string jika panjangnya melebihi batas buffer. Namun, karena itu hanya peringatan, proses kompilasi tetap berhasil dan menghasilkan file eksekusi `sysinfo`.
-

3. Proses Eksekusi

- Setelah berhasil dikompilasi, program dijalankan dengan perintah:
- `./sysinfo`
- Tanda `./` berarti menjalankan program dari direktori saat ini (bukan dari PATH global).
- Saat dijalankan, program akan mulai membaca file-file dari direktori `/proc` seperti:
 - `/proc/cpuinfo` → untuk mendapatkan model dan jumlah core CPU.
 - `/proc/meminfo` → untuk membaca total dan free memory.
 - `/proc/stat` atau `/proc/[pid]/comm` → untuk menampilkan proses yang sedang berjalan.

4. Hasil Keluaran Program

- Setelah program dijalankan, muncul tampilan seperti berikut:
- Program menampilkan data sistem secara **real-time** dengan format terstruktur dan mudah dibaca.

5. Penjelasan Mekanisme Program

- Program bekerja dengan **membaca file virtual** dari sistem Linux, bukan dari perangkat keras langsung.
- Fungsi seperti `fopen()` digunakan untuk membuka file di `/proc`, lalu `fgets()` untuk membaca isi baris demi baris.
- Hasil pembacaan disimpan dalam variabel dan ditampilkan dengan `printf()`.
- Selain itu, program juga mendeteksi proses yang sedang berjalan dengan membaca setiap direktori di `/proc` yang berisi angka (yang mewakili PID).

BAB III

LAMPIRAN

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <dirent.h>
```

```
#include <sys/statvfs.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#define BUF_SIZE 4096
```

```
// --- CPU Info ---
```

```
void bacaCPUInfo() {
```

```
    printf("=== CPU INFO ===\n");
```

```
    int fd = open("/proc/cpuinfo", O_RDONLY);
```

```
    if (fd < 0) {
```

```
        perror("Gagal membuka /proc/cpuinfo");
```

```
        return;
```

```
    }
```

```
    char buffer[BUF_SIZE];
```

```
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
```

```
    buffer[bytes] = '\0';
```

```
    close(fd);
```

```
    char *line = strtok(buffer, "\n");
```

```
    while (line) {
```

```
        if (strstr(line, "model name") || strstr(line, "cpu cores")) {
```

```
            printf("%s\n", line);
```

```
        }
```

```
        line = strtok(NULL, "\n");
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
// --- Memory Info ---
```

```
void bacaMemori() {
```

```
    printf("=== MEMORY USAGE ===\n");
```

```

FILE *fp = fopen("/proc/meminfo", "r");
if (!fp) {
    perror("Gagal membuka /proc/meminfo");
    return;
}

char label[64];
long value;
while (fscanf(fp, "%s %ld kB\n", label, &value) == 2) {
    if (strcmp(label, "MemTotal:") == 0)
        printf("Total Memory : %ld kB\n", value);
    if (strcmp(label, "MemFree:") == 0) {
        printf("Free Memory : %ld kB\n", value);
        break;
    }
}
fclose(fp);
printf("\n");
}

// --- Disk Usage ---
void bacaDiskUsage() {
    printf("=== DISK USAGE ===\n");
    struct statvfs fs;
    if (statvfs("/", &fs) != 0) {
        perror("statvfs gagal");
        return;
    }
    unsigned long total = fs.f_blocks * fs.f_frsize / 1024 / 1024;
    unsigned long free = fs.f_bfree * fs.f_frsize / 1024 / 1024;

```

```

unsigned long used = total - free;

printf("Total: %lu MB\nUsed : %lu MB\nFree : %lu MB\n\n",
      total, used, free);
}

// --- Process Info ---
void bacaProcess() {
    printf("=== RUNNING PROCESSES ===\n");
    DIR *dir = opendir("/proc");
    struct dirent *entry;

    if (!dir) {
        perror("Gagal membuka /proc");
        return;
    }

    int count = 0;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_type == DT_DIR && atoi(entry->d_name) > 0) {
            char path[256];
            snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);

            FILE *f = fopen(path, "r");
            if (f) {
                char nama[128];
                fgets(nama, sizeof(nama), f);
                nama[strcspn(nama, "\n")] = '\0';
                printf("PID: %s - %s\n", entry->d_name, nama);
                fclose(f);
            }
        }
    }
}

```



```

        if(++count >= 10) break; // tampilkan 10 proses saja
    }
}
}
closedir(dir);
printf("\n");
}

// --- Network Info ---
void bacaNetwork() {
    printf("=== NETWORK INTERFACES ===\n");
    FILE *fp = fopen("/proc/net/dev", "r");
    if(!fp) {
        perror("Gagal membuka /proc/net/dev");
        return;
    }

    char line[256];
    int skip = 2; // lewati header
    while (fgets(line, sizeof(line), fp)) {
        if (skip-- > 0) continue;

        char iface[32];
        unsigned long rx, tx;
        sscanf(line, "%[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %lu",
            iface, &rx, &tx);
        printf("Interface: %-10s RX: %lu bytes, TX: %lu bytes\n", iface, rx, tx);
    }
    fclose(fp);
    printf("\n");
}

```

```

}

// --- TCP Socket Info (untuk NIM GANJIL) ---
void bacaTCPSocket() {
    printf("=== NETWORK SOCKET INFO (/proc/net/tcp) ===\n");

    FILE *fp = fopen("/proc/net/tcp", "r");
    if(!fp) {
        perror("Gagal membuka /proc/net/tcp");
        return;
    }

    char line[512];
    fgets(line, sizeof(line), fp); // lewati header

    int count = 0;
    while (fgets(line, sizeof(line), fp)) {
        unsigned int local_addr, rem_addr;
        int local_port, rem_port, state;

        sscanf(line, "%*d: %x:%x %x:%x %x",
            &local_addr, &local_port, &rem_addr, &rem_port, &state);

        printf("Socket %d:\n", ++count);
        printf("  Local Addr : %08X:%04X\n", local_addr, local_port);
        printf("  Remote Addr: %08X:%04X\n", rem_addr, rem_port);
        printf("  State      : %02X\n\n", state);

        if (count >= 5) break; // tampilkan 5 socket pertama
    }
}

```

```
fclose(fp);  
printf("\n");  
}  
  
int main() {  
    printf("=====\n");  
    printf(" SYSTEM INFORMATION TOOL (NIM GANJIL)\n");  
    printf("=====\n\n");  
  
    bacaCPUInfo();  
    bacaMemori();  
    bacaDiskUsage();  
    bacaProcess();  
    bacaNetwork();  
    bacaTCPSocket(); // Ganti ini karena NIM kamu GANJIL  
  
    printf("Selesai menampilkan informasi sistem.\n");  
    return 0;  
}
```

BAB IV

PENUTUP

Kesimpulan

Dari seluruh rangkaian praktikum Sistem Operasi mulai dari Tugas 1 hingga Tugas 4, dapat disimpulkan bahwa kegiatan ini membantu memahami konsep dasar kerja sistem operasi melalui implementasi langsung menggunakan bahasa C. Pada Tugas 1, mahasiswa belajar tentang manajemen proses dan komunikasi antarproses (IPC). Tugas 2 memperkenalkan mekanisme monitoring file menggunakan inotify. Tugas 3 berfokus pada pembuatan custom *shell* yang meniru fungsi dasar shell Linux, sementara Tugas 4 memperdalam pemahaman mengenai struktur direktori `/proc` untuk menampilkan informasi sistem. Dengan praktikum ini, mahasiswa mampu memahami konsep process management, interprocess *communication*, file system monitoring, serta system information handling secara lebih konkret.

Saran

Pelaksanaan praktikum ini sangat bermanfaat dalam memperkuat pemahaman teori sistem operasi. Namun, ke depan disarankan agar diberikan tambahan latihan mengenai manajemen memori dan penjadwalan proses agar mahasiswa dapat memahami aspek yang lebih luas dari sistem operasi. Selain itu, penggunaan contoh kasus yang lebih kompleks akan membantu mahasiswa mengasah kemampuan analisis dan penerapan konsep sistem operasi dalam konteks dunia nyata.